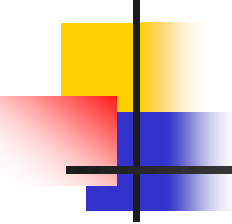


OOP JAVA PROGRAMMING



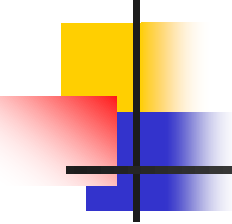
Introduction

- Java programming language was originally developed by Sun Microsystems which was initiated by James Gosling and released in 1995 as core component of Sun Microsystems' Java platform (Java 1.0 [J2SE]).
- The latest release of the Java Standard Edition is Java SE 8. With the advancement of Java and its widespread popularity, multiple configurations were built to suit various types of platforms.
- For example: J2EE for Enterprise Applications, J2ME for Mobile Applications.
- The new J2 versions were renamed as Java SE, Java EE, and Java ME respectively. Java is
- guaranteed to be Write Once, Run Anywhere.
- Java is:
 - *Object Oriented*: In Java, everything is an Object. Java can be easily extended since it is based on the Object model.
 - *Platform Independent*: Unlike many other programming languages including C and C++, when Java is compiled, it is not compiled into platform specific machine, rather into platform independent byte code. This byte code is distributed over the web and interpreted by the Virtual Machine (JVM) on whichever platform it is being run on.
 - *Simple*: Java is designed to be easy to learn. If you understand the basic concept of OOP Java, it would be easy to master.



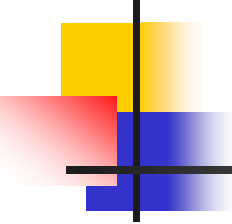
Strengths of JAVA

- *Secure:* With Java's secure feature it enables to develop virus-free, tamper-free systems.
- Authentication techniques are based on public-key encryption.
- *Architecture-neutral:* Java compiler generates an architecture-neutral object file format, which makes the compiled code executable on many processors, with the presence of Java runtime system.
- *Portable:* Being architecture-neutral and having no implementation dependent aspects of the specification makes Java portable. Compiler in Java is written in ANSI C with a clean portability boundary, which is a POSIX subset.
- *Robust:* Java makes an effort to eliminate error prone situations by emphasizing mainly on compile time error checking and runtime checking.
- *Multithreaded:* With Java's multithreaded feature it is possible to write programs that can perform many tasks simultaneously. This design feature allows the developers to construct interactive applications that can run smoothly.
- *Interpreted:* Java byte code is translated on the fly to native machine instructions and is not stored anywhere. The development process is more rapid and analytical since the linking is an incremental and light-weight process.
- High Performance: With the use of Just-In-Time compilers, Java enables high performance.
- Distributed: Java is designed for the distributed environment of the internet.
- Dynamic: Java is considered to be more dynamic than C or C++ since it is designed to adapt to an evolving environment. Java programs can carry extensive amount of run-time information that can be used to verify and resolve accesses to objects on run-time.



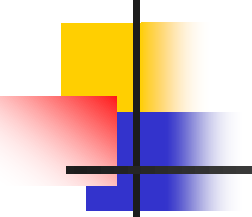
History of Java

- James Gosling initiated Java language project in June 1991 for use in one of his many set-top box projects. The language, initially called 'Oak' after an oak tree that stood outside Gosling's office, also went by the name 'Green' and ended up later being renamed as Java, from a list of random words.
- Sun released the first public implementation as Java 1.0 in 1995. It promised Write Once, Run Anywhere (WORA), providing no-cost run-times on popular platforms.
- On 13 November, 2006, Sun released much of Java as free and open source software under the terms of the GNU General Public License (GPL).
- On 8 May, 2007, Sun finished the process, making all of Java's core code free and open-source, aside from a small portion of code to which Sun did not hold the copyright.



Java Concepts

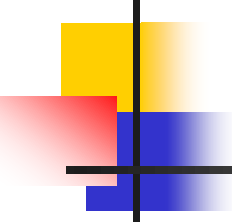
- Object - Objects have states and behaviors. Example: A dog has states - color, name, breed as well as behavior such as wagging their tail, barking, eating. An object is an instance of a class.
- Class - A class can be defined as a template/blueprint that describes the behavior/state that the object of its type supports.
- Methods - A method is basically a behavior. A class can contain many methods. It is in methods where the logics are written, data is manipulated and all the actions are executed.
- Instance Variables - Each object has its unique set of instance variables. An object's state is created by the values assigned to these instance variables.



BASICS OF JAVA PROGRAM

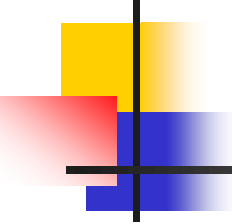
■ **JAVA TOKENS**

- Java Tokens are the smallest individual building block or smallest unit of a Java program; the Java compiler uses it for constructing expressions and statements. Java program is a collection of different types of tokens, comments, and white spaces.
- A java program is basically a collection of classes.
- A class is defined by a set of declaration statements and methods containing executable statements
- Most statements contain expressions, which describe the actions carried out on data.
- Smallest units in a program are known as tokens.
- In short, a java program is a collection of tokens, comments and white spaces.
- Java includes following tokens:
 - Reserved keywords
 - Expressions
 - Identifiers
 - Literals
 - Operators
 - Separators



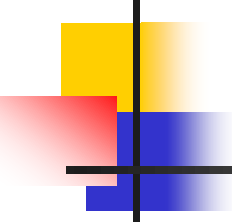
Java Keywords

- Keywords are words that have already been defined for Java compiler. They have special meaning for the compiler. Java Keywords must be in your information because you cannot use them as a variable, class or a method name. Examples: break, byte, case, catch, char
- Java Expressions
- A Java expression is a combination of variables, literal values and operators, ending with a semicolon.



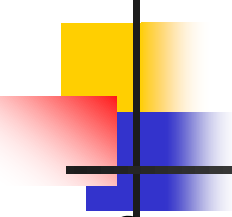
Identifiers

- They are programmer-designed names of variables, classes, methods, etc.
- They are used for naming classes, methods, variables, objects, labels, packages and interfaces in a program.
- It follows the following rules:
 - They can have alphabets, digits and underscore and dollar sign characters.
 - They must not begin with digit.
 - Uppercase and Lowercase letters are distinct.
 - They can not contain spaces



Literals

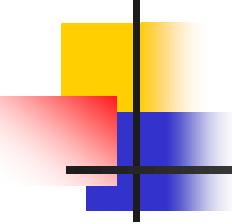
- Literals in Java are sequence of characters that represent constant values to be stored in variables.
- Java supports five types of literals:
 - Integer literals
 - Floating-point literals
 - Character literals
 - String literals
 - Boolean literals



Separators

- Separators are symbols that are used to indicate where groups of code are divided and arranged

Name	Use
parentheses()	Used to enclose parameters in method definition, also used for defining precedence in expressions.
braces{ }	Used to contain the values of automatically initialized arrays and to define a block of code classes, methods.
Brackets []	Used to declare array types and for dereferencing values.
Semicolon ;	Used to separate statements.
Comma ,	Used to separate consecutive identifiers in a variable declaration.
period .	Used to separate package names from sub-packages and classes.

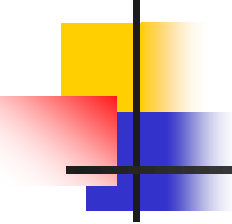


Data Types in Java

- **Integer type**
- Used to hold whole numbers such as 8, 10, 12, etc.

Example

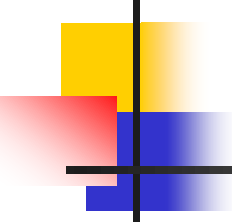
```
public static void main(String[ ] args)
{
    int first_number;
    System.out.println("My First Project");
}
```



Double

- The double variable can hold very large (or small) numbers. The maximum and minimum values are 17 followed by 307 zeros. The double variable is also used to hold floating point values. A floating point value is one like 8.7, 12.5, 10.1. In other words, it has a "point something" at the end.
- Example: double first_number, second_number, answer;

```
■ public static void main(String[] args) {  
  
    double first_number, second_number, answer;  
  
    first_number = 10.5;  
    second_number = 20.8;  
    answer = first_number + second_number;  
  
    System.out.println("Addition Total = " + answer );  
}
```



Float

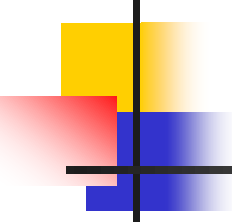
- A float data type in Java stores a decimal value with 6-7 total digits of precision. It is used when you want to save memory and when calculations don't require more than 6 or 7 digits of precision.
- Two more variable types you can use are **short** and **float**. The short variable type is used to store smaller number, and its range is between minus 32,768 and plus 32,767. You should only use short if you're sure that the values that you want to hold don't go above 32, 767 or below -32,768.
- When storing a value in a float variable, you need the letter "f" at the end. Like this:
- Example

```
float first_number, second_number, answer;
```

```
first_number = 10.5f;
```

```
second_number = 20.8f;
```

So the letter "f" goes after the number but before the semicolon at the end of the line.



Character (char)

- This is used to store just a single character. To store the character, you use single quotes instead of double quotes

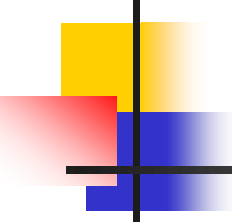
```
public static void main(String[] args) {
```

```
    char first_name = 'W';
```

```
    char family_name = 'S';
```

```
    System.out.println(first_name + " " + family_name);
```

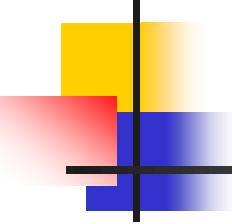
```
}
```



String Variables

- This is used to store text data. Usually, used when you want to store more than one character. To do so, you need the string variable type.

```
public static void main(String[] args) {  
  
    String first_name = "William";  
    String family_name = "Shakespeare";  
  
    System.out.println(first_name + " " + family_name);  
}
```

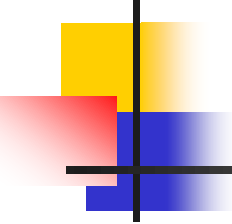


Variable Declaration

- Java is a “declarative” language.
- In other words, variables must be declared before they can be used.
- To declare a variable, use the Java “keyword” appropriate for the type of variable you are declaring followed by a variable name you have created, followed by a semicolon.

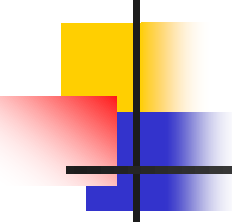
Examples:

- `int aNum;`
- `double totalVolume;`
- `String userPrompt;`



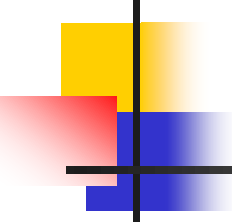
Rules in Declaring Variable Names

- Variable names can't start with a number. Java names may contain any number of letters, numbers and underscore ("_") characters, but they must begin with a letter. Therefore, `first_number` is OK, but not `1st_number`. You can have numbers elsewhere in the variable name, just not at the start.
- Variable names can't be the same as Java keywords. There are quite a lot of these, and they will turn blue in NetBeans, like `int` above.
- You can't have spaces in your variable names. The variable declaration *`int first number`* will get you an error. We've used the underscore character, but it's common practise to have the first word start with a lowercase letter and the second or subsequent words in uppercase: `firstNumber`, `myFirstNumber`.
- Variable names are case sensitive. So `firstNumber` and `FirstNumber` are different variable names.



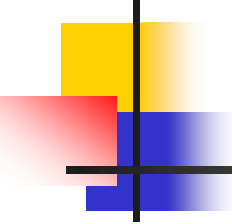
Standard Java Naming Convention:

- Names beginning with lowercase letters are variables or methods.
 - Names beginning with uppercase letters are class names.
 - Successive words within a name are capitalized.
 - Names in all capital letters are constants.
-
- int and double variables initially are given a value of zero unless they are initialized to a value.
 - Java may prevent you from using variables that are not initialized.
 - So, it is often good practice to initialize your variables before use, for example:
 -
 - `int numDaysInYear = 365;`
 - `double avgNumDaysInYear = 365.25;`
 - `String greetingLine = "Hello there!";`



Constants

- The Java keyword, “final” can be used to make sure a variable value is no longer “variable”.
- It becomes a constant, because Java will not allow your program to change its value once it has been declared:
- `final int NUM_DAYS_IN_YEAR = 365;`
- `final double MM_PER_INCH = 25.4;`
-

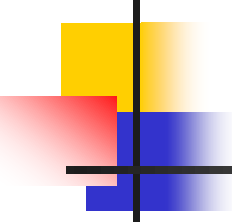


OPERATORS IN JAVA

- **Arithmetic Operators**

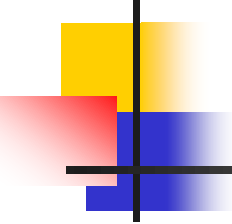
- The standard arithmetic operators in Java are:
 - Addition (+)
 - Subtraction (-)
 - Multiplication (*)
 - Division (/)
 - Modulus or Remainder (%) (ie. 12 % 5 yields 2)
- All of these operations apply to all numeric data types.
- All require values on both sides of the operator





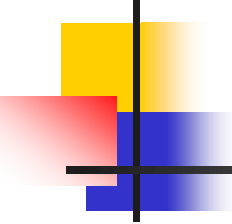
Unary Arithmetic Operators

- Unary operators include “+” and “-”, where “-aNum” negates the value produced by aNum, for example.
- They also include the increment (++) and decrement (--) operators, which increase or decrease an integer value by 1.
- Preincrement and predecrement operators appear before a variable. They increment or decrement the value of the variable before it is used in the expression.
- **Example:**
- ```
Int i = 4, j = 2, k;
k = ++i - j; // i = 5, j = 2, k = 3
```
- Postincrement and postdecrement operators appear after a variable. They increment or decrement the value of the variable after it is used in the expression.
- **Example:**
- ```
int i = 4, j = 2, k;  
k = i++ - j;      // i = 5, j = 2, k = 2
```
- Keep expressions involving increment and decrement operators simple!



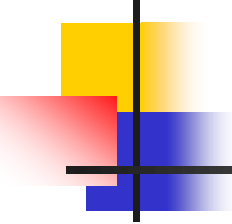
Assignment Operators

- `=` set equal to
- `*=` multiply and set equal to
- `/=` divide and set equal to
- `-=` subtract and set equal to
- `+=` add and set equal to
-
- An assignment statement in Java has the form
`variableName = expression;`
- The expression is evaluated, and the result of the expression is stored in the specified variable.
- Assignment statements and arithmetic operations can be combined in a single symbol.
- For example,
`variableName += expression;` is equivalent to `variableName = variableName + expression;`



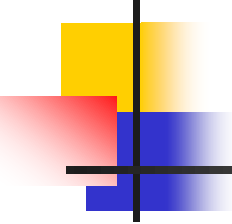
Logical Binary Operators

- Return either true or false.
 - == equals to
 - != not equals to
 - > greater than
 - < less than
 - >= greater than or equal to
 - <= less than or equal to
 - &, && logical "And"
 - |, || logical "Or"
- The one "unary" logical operator is "!". Called the "Not" operator.
- It reverses the logical value of a boolean.
- For example:
- !(5 > 3) evaluates to false
-



Precedence Rules

- Operator precedence rules determine which operations take place in what order:
 - Unary operations are done first.
 - Then $*$, $/$, $\%$
 - Then $+$, $-$
 - Then $<$, $>$, $\leq$, $\geq$
 - Then $==$, $!=$
 - Then $\&$, $\&\&$, $|$, $||$
 - Then $=$, $*=$, $+=$, $-=$, $/=$
- Use “()” to control order of operations, as the expression inside “()” will be evaluated before stuff outside of “()”.



Relational operators

- The Java Relational operators compare between operands and determine the relationship between them.
- There are six types of relational operators in Java, these are:

== Is equal to

!= Is not equal to

> Greater than

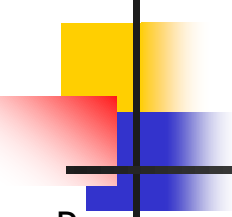
< Less than

>= Greater than or equal to

<= Less than or equal to

■

- Program example

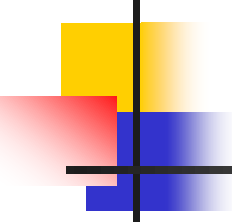


Relational operator Program example

Program example

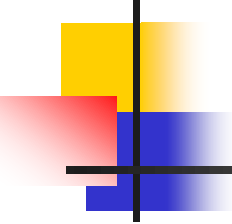
//this program displays use of relational operators

```
public class relatiop {  
    public static void main(String[] args) {  
        //Variables Definition and Initialization  
        int num1 = 12, num2 = 4;  
        //is equal to  
        System.out.println("num1 == num2 = " + (num1 == num2) );  
        //is not equal to  
        System.out.println("num1 != num2 = " + (num1 != num2) );  
        //Greater than  
        System.out.println("num1 > num2 = " + (num1 > num2) );  
        //Less than  
        System.out.println("num1 < num2 = " + (num1 < num2) );  
        //Greater than or equal to  
        System.out.println("num1 >= num2 = " + (num1 >= num2) );  
        //Less than or equal to  
        System.out.println("num1 <= num2 = " + (num1 <= num2) ); }  
}
```



Class Libraries

- A class library is a collection of classes that we can use when developing programs
- There is a Java standard class library that is part of any Java development environment
- These classes are not part of the Java language per se, but we rely on them heavily
- The **System** class and the **String** class are part of the Java standard class library
- Other class libraries can be obtained through third party vendors, or you can create them yourself



Packages

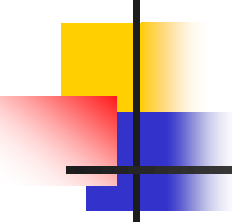
- The classes of the Java standard class library are organized into packages
- Some of the packages in the standard class library are:

Package

java.lang
java.applet
java.awt
javax.swing
java.net
java.util

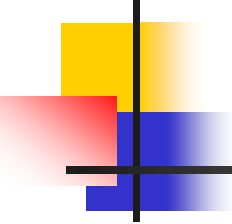
Purpose

General support
Creating applets for the web
Graphics and graphical user interfaces
Additional graphics capabilities and components
Network communication
Utilities



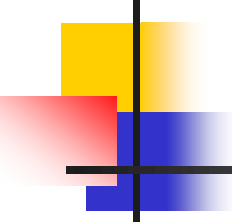
The import Declaration

- When you want to use a class from a package, you could use its fully qualified name : `java.util.Random`
- Or you can import the class, then just use the class name
`import java.util.Random;`
- To import all classes in a particular package, you can use the * wildcard character : `import java.util.*;`
- All classes of the `java.lang` package are automatically imported into all programs
- That's why we didn't have to explicitly import the `System` or `String` classes in earlier programs
- The `Random` class is part of the `java.util` package
- It provides methods that generate pseudo-random numbers
- We often have to scale and shift a number into an appropriate range for a particular purpose



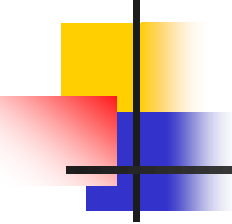
Class Methods

- Methods cannot exist on their own - they must “belong” to a class.
- In order for Java to find the method, you must tell Java the name of the class that “owns” the method.
- Some methods can be invoked through the class name, instead of through an object of the class
- These methods are called *class methods or static* methods
- The Math class contains many static methods, providing various mathematical functions, such as absolute value, trigonometry functions, square root, etc.
- `temp = Math.cos(90) + Math.sqrt(delta);`
-



Dot Operator

- Telling the Java compiler who owns the method is done by using the “dot operator”.
- The period is used to separate the class name from the method name:
- For example: `StdIn.readInt()`
- If a class exists within another class, then you get a “chain” of periods:
-
- For example: `System.out.println()`
- (However, if you are calling a method that exists within the same class as your main method, you do not need a class name or a period.

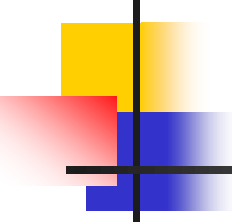


Escape Sequence

- String's can contain special characters that can control screen output:

<code>\n</code>	positions the cursor at the beginning of the next line
<code>\t</code>	inserts a tab
<code>\r</code>	positions the cursor at the beginning of the current line
<code>\\</code>	inserts "\" as a character
<code>\'</code>	inserts a single quote as a character
<code>\"</code>	inserts a double quote as a character

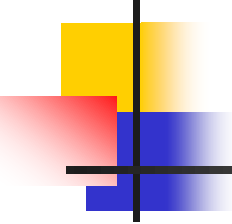
- For example the following code:
- **`System.out.println("\Tabbed\ output:\t1\t2\t3\t4");`**
- Prints the following to the screen:
-
- **"Tabbed" output: 1 2 3 4**



First Java Program

- Let us look at a simple code that will print the words ***Hello World***.

```
public class MyFirstJavaProgram {  
    /* This is my first java program.  
    * This will print 'Hello World' as the output  
    */  
    public static void main(String []args) {  
        System.out.println("Hello World"); // prints Hello World  
    }  
}
```



COMMENTS IN JAVA

- Java supports single-line and multi-line comments very similar to C and C++. All characters available inside any comment are ignored by Java compiler.

```
public class MyFirstJavaProgram{  
    /* This is my first java program.  
    * This will print 'Hello World' as the output  
    * This is an example of multi-line comments. */  
    public static void main(String []args){  
        // This is an example of single line comment  
        /* This is also an example of single line comment. */  
        System.out.println("Hello World");  
    }  
}
```

- }